CENG 331 Performance Lab

Recitation: Cache Optimization Techniques

Computer Organization

Middle East Technical University
Department of Computer Engineering

Fall 2025-2026

Outline

1. Introduction
2. Bilateral Filter
3. Box Blur
4. Cache Optimization Techniques
   - Code Motion
   - Avoiding Costly Operations
   - Loop Unrolling
   - Reducing Sequential Dependency
   - Cache-Friendly Code: Spatial Locality
   - Cache-Friendly Code: Temporal Locality & Blocking
5. Optimization Strategies
6. Performance Measurement
7. Best Practices
8. Summary

Objectives

- Optimize memory-intensive image processing code
- Learn cache optimization techniques
- Apply techniques to two operations:
  - **Bilateral Filter** (70 points) - Complex, scaled speedup
  - **Box Blur** (30 points) - Simpler, flat 1.3x threshold
- Understand how cache behavior affects performance

Image Representation

- Image: 2D matrix M where $M_{i,j}$ is pixel at position (i, j)
- Pixel: RGB triple (red, green, blue)
- Square images: $N \times N$ where N is dimension
- Stored as 1D array: `I[RIDX(i,j,n)]`
- `RIDX(i,j,n) = (i)*(n) + (j)`

Data Structure

```
typedef struct {
    unsigned short red;
    unsigned short green;
    unsigned short blue;
} pixel;
```

Bilateral Filter Overview

- Non-linear, edge-preserving image filtering
- Smooths images while preserving sharp edges
- Considers both **spatial proximity** and **color similarity**
- Memory-intensive: accesses many neighboring pixels

Bilateral Filter Formula

For pixel at position (i, j):

$$M_{i,j} = \frac{1}{W_{i,j}} \sum_{u=-K}^{K} \sum_{v=-K}^{K} S_{i+u,j+v} \cdot w_s(u, v) \cdot w_r(S_{i,j}, S_{i+u,j+v})$$

Spatial weight (distance-based):

$$w_s(u, v) = \exp\left(-\frac{u^2 + v^2}{2\sigma_s^2}\right)$$

Range weight (color similarity):

$$w_r(S_{i,j}, S_{i+u,j+v}) = \exp\left(-\frac{\|S_{i,j} - S_{i+u,j+v}\|^2}{2\sigma_r^2}\right)$$

Bilateral Filter: Key Challenges

- **Poor cache locality**: Accesses many different image regions
- **Expensive computations**: Multiple `exp()` calls per pixel
- **Memory access patterns**: Non-sequential access to source image
- **Solution**: Cache blocking + precomputed weights

Box Blur Overview

- Simple averaging filter
- Computes average of pixels in square window
- Creates smoothing effect
- Simpler than bilateral (no exponential calculations)
- Still memory-intensive: 5x5 window per pixel

Box Blur Formula

For pixel at position (i, j):

$$M_{i,j} = \frac{1}{(2K+1)^2} \sum_{u=-K}^{K} \sum_{v=-K}^{K} S_{i+u,j+v}$$

- $K = 2$ (radius) $\Rightarrow$ 5x5 window
- Window area $= (2K+1)^2 = 25$ pixels
- Mirror padding at boundaries
- **Challenge**: Cache locality when accessing neighbors

Cache Optimization Overview

1. **Code Motion** - Move computations out of loops
2. **Avoiding Costly Operations** - Reduce expensive calculations
3. **Reducing Sequential Dependency** - Enable parallelism
4. **Loop Unrolling** - Reduce loop overhead
5. **Cache-Friendly Code** - Improve spatial/temporal locality
   - Spatial locality: Access nearby memory locations
   - Temporal locality: Reuse recently accessed data
   - Blocking/Tiling: Process data in cache-sized blocks

Code Motion

Principle: Move invariant computations outside loops

Before

```
for (i = 0; i < N; i++) {
    for (j = 0; j < N; j++)
        a[N*i+j] = b[j];
}
```

After

```
for (i = 0; i < N; i++) {
    ni = N*i;
    for (j = 0; j < N; j++)
        a[ni+j] = b[j];
}
```

- `N*i` computed once per outer iteration
- Reduces redundant multiplications
- **For bilateral/blur**: Precompute `two_sigma_s_sq`, `window_area`

Avoiding Costly Operations

Principle: Replace expensive operations with cheaper alternatives

Before

```
for (i = 0; i < N; i++) {
    ni = N*i;
    for (j = 0; j < N; j++)
        a[ni+j] = b[j];
}
```

After

```
ni = 0;
for (i = 0; i < N; i++) {
    for (j = 0; j < N; j++)
        a[ni+j] = b[j];
    ni += N;  // Addition instead of multiplication
}
```

- Addition is cheaper than multiplication
- **For bilateral**: Precompute spatial weights in lookup table
- **For blur**: Precompute `window_area` once

Loop Unrolling

Principle: Reduce loop overhead by processing multiple iterations

Before

```c
int i, sum;
sum = 0;
for (i = 0; i < N; i++) {
    sum += a[i];
}
```

After

```c
int i, sum;
sum = 0;
for (i = 0; i < N; i+=2) {
    sum += a[i] + a[i+1];
}
```

- Reduces loop control overhead
- Enables better instruction scheduling
- **For blur**: Unroll inner v loop (5 iterations: -2, -1, 0, 1, 2)
- **For bilateral**: Unroll radius loops when possible

Reducing Sequential Dependency

Principle: Break data dependencies to enable parallelism

Before

```
int i, sum;
sum = 0;
for (i = 0; i < N; i+=2) {
    sum += a[i] + a[i+1];
}
```

After

```
int i, sum, s1, s2;
s1 = 0; s2 = 0;
for (i = 0; i < N; i+=2) {
    s1 += a[i];        // Independent
    s2 += a[i+1];      // Independent
}
sum = s1 + s2;
```

- Two independent accumulators can execute in parallel
- CPU can pipeline operations
- **For blur**: Separate RGB accumulators (already done)
- **For bilateral**: Separate spatial/range weight calculations

Spatial Locality

Principle: Access memory locations that are close together

Poor Locality

```
for (i = 0; i < N; i++) {
    for (j = 0; j < N; j++)
        sum += a[j][i];  // Column-
            major
}
```

Good Locality

```
for (i = 0; i < N; i++) {
    for (j = 0; j < N; j++)
        sum += a[i][j];  // Row-
            major
}
```

- Row-major access: sequential memory access
- Column-major: cache misses on every access
- **For blur/bilateral**: Process pixels row by row

Temporal Locality & Blocking

Principle: Reuse data while it's still in cache

Blocking/Tiling Example

Process matrix multiplication in blocks that fit in cache:

```
for (i = 0; i < N; i+=B) {
    for (j = 0; j < N; j+=B) {
        for (k = 0; k < N; k+=B) {
            // Process BxB block
            for (i1 = i; i1 < i+B; i1++) {
                for (j1 = j; j1 < j+B; j1++) {
                    for (k1 = k; k1 < k+B; k1++) {
                        c[i1*N+j1] += a[i1*N+k1] * b[k1*N+j1];
                    }
                }
            }
        }
    }
}
```

Blocking for Image Filters

For Bilateral Filter and Box Blur:

- Process image in **8x8 or 16x16 blocks**
- Keep frequently accessed pixels in cache
- Reduces cache misses when accessing neighboring pixels
- Block size should fit in L1 cache

Blocking Structure

```
for (ii = 0; ii < dim; ii += BLOCK_SIZE) {
    for (jj = 0; jj < dim; jj += BLOCK_SIZE) {
        for (i = ii; i < ii+BLOCK_SIZE && i < dim; i++) {
            for (j = jj; j < jj+BLOCK_SIZE && j < dim; j++) {
                // Process pixel (i,j) with neighbors
            }
        }
    }
}
```

Bilateral Filter Optimization Strategy

1. **Precompute spatial weights**
   - Spatial weights depend only on (u, v)
   - Create lookup table: `spatial_weights[u+2][v+2]`
   - Avoid repeated `exp()` calls
2. **Cache blocking (8x8 or 16x16)**
   - Process image in blocks
   - Keep source pixels in cache
3. **Register optimization**
   - Cache center pixel RGB values
   - Reuse for range weight calculations
4. **Loop unrolling**
   - Unroll inner v loop (5 iterations)
   - Reduce loop overhead

Box Blur Optimization Strategy

1. **Cache blocking (8x8)**
   - Process image in 8x8 blocks
   - Improve spatial locality
2. **Loop unrolling**
   - Unroll inner v loop completely (5 iterations)
   - Eliminate inner loop overhead
3. **Register usage**
   - Cache index calculations
   - Reuse boundary check results
4. **Code motion**
   - Precompute `window_area` once
   - Move invariant calculations outside loops

CPE: Cycles Per Element

Definition

If a function takes C cycles for an image of size $N \times N$:

$$\text{CPE} = \frac{C}{N^2}$$

- Lower CPE = better performance
- Independent of image size
- Allows fair comparison across different dimensions

Speedup

$$\text{Speedup} = \frac{\text{Naive CPE}}{\text{Optimized CPE}}$$

Performance Example

Method	**64**	**128**	**256**	**512**	**1024**	**Mean**
Bilateral Filter						
Naive (CPE)	1442.9	1440.9	1447.5	1448.6	1451.9	
Optimized (CPE)	907.3	906.9	910.1	911.1	913.6	
Speedup	**1.6**	**1.6**	**1.6**	**1.6**	**1.6**	**1.6**
Box Blur						
Naive (CPE)	103.6	103.5	103.5	103.8	104.5	
Optimized (CPE)	71.9	72.0	72.4	72.2	72.9	
Speedup	**1.4**	**1.4**	**1.4**	**1.4**	**1.4**	**1.4**

Performance Targets

Function	Threshold	Points
Bilateral Filter	1.3x	40 (scaled to 70)
Box Blur	1.3x	30 (flat)

- **Bilateral**: Scaled scoring (40-70 points)
  - 40 points for 1.3x speedup (threshold)
  - 70 points for best performance in class
  - Linear scaling between 1.3x and best speedup
  - Example: Optimized achieves 1.3x speedup
- **Blur**: Flat threshold
  - 30 points if speedup $\geq$ 1.3x
  - 0 points if speedup $<$ 1.3x
  - Example: Optimized achieves 1.4x speedup

Best Practices

1. **Start with correctness**
   - Make sure your code works correctly first
   - Use driver's correctness checks
2. **Measure, don't guess**
   - Run `./driver` to see actual performance
   - Compare different versions
3. **Use versioning**
   - Register multiple versions
   - Compare side-by-side
   - Keep what works, discard what doesn't
4. **Test multiple times**
   - Performance varies with CPU state
   - Take best results

Common Pitfalls

- **Modifying input**: Only modify `dst`, not `src`
- **Array bounds**: Don't exceed image boundaries
- **Printing output**: Don't print anything (affects timing)
- **Assembly code**: ANSI C only, no inline assembly
- **Compiler flags**: Don't modify Makefile compiler flags
- **Team info**: Fill in team struct immediately!

Key Takeaways

- **Cache optimization** is crucial for memory-intensive code
- **Blocking/tiling** improves cache locality
- **Loop unrolling** reduces overhead
- **Code motion** eliminates redundant calculations
- **Register usage** reduces memory accesses
- **Measure performance** to validate optimizations

Resources

- **Textbook**: CS:APP Chapter 5 (Optimizing Program Performance)
- **Driver**: `./driver -h` for help
- **Testing**: `./driver -g` for autograder mode
- **Deadline**: January 8, 2026, 23:59
- **Submission**: ODTUClass, only `kernels.c`

Questions?